pyspider-web

Plugin Guide

How to extend the node catalog
and file-format support without touching core files.

Version 0.1.0 · 2026-08-31

1. What is a plugin?

A plugin is a single Python file that adds one or more nodes (operations that appear in the palette) and/or file-format readers/writers to pyspider-web. No changes to the core files are needed — drop the file into a plugin directory, restart the backend, and the palette gains a new entry.

Discovery

On backend startup, plugins are discovered in this order:

- backend/plugins/ — bundled with the distribution
- $PYSPIDER_PLUGIN_DIR entries (colon-separated on macOS/Linux)
- ~/.pyspider/plugins/ — user-scope, survives app upgrades

Every non-private .py file (files starting with _ are skipped) is imported once. During import, plugin code calls register(...) to contribute nodes and register_reader / register_writer to contribute file formats. Failed plugins do not stop the rest of the catalog from loading — check GET /api/plugins for status per file.

Design contract

The only stable interface for plugins is backend/plugin_api.py. Every helper you need — NodeSpec, N, h, d, register, register_reader, register_writer, xp — is re-exported from that module. Never import from backend.nodes directly; the internals may reorganise between versions.

2. Minimum node — Sobel edge

This example ships with the distribution as backend/plugins/example_edge.py. Read the file for the exact text; a slightly abbreviated version:

```
"""Example plugin — Sobel edge magnitude in a Custom category."""

import numpy as np
from scipy import ndimage as ndi
from backend.plugin_api import register, N, h, d

def _fn_sobel(image, sigma: float = 1.0):
    a = h(image).astype(np.float32)
    if sigma > 0:
        a = ndi.gaussian_filter(a, float(sigma))
    gx = ndi.sobel(a, axis=1)
    gy = ndi.sobel(a, axis=0)
    return {"image": d(np.hypot(gx, gy).astype(np.float32))}

register(
    N("plugin.sobel", "Sobel edge (plugin)", "Custom",
      "image:image", "image:image",
      [("sigma", "float", 1.0, None, 0.0, 20.0)],
      "Gaussian-smoothed Sobel gradient magnitude.",
      _fn_sobel),
)
```

Anatomy

- N(id, label, category, inputs, outputs, params, doc, fn) — factory that builds a NodeSpec. Prefer this over constructing NodeSpec manually; keeps declarations short.
- id must be globally unique. Collisions with a built-in are silently rejected (built-in wins). Prefix with 'plugin.' or your handle to avoid clashes.
- h(x) → move x to CPU (host); d(x) → move x to the current backend (numpy or cupy). Use these at input/output boundaries so your plugin transparently supports CPU/GPU switching.

3. Node schema reference

Port kinds (strings)

- "image"   — 2D numpy array (ny, nx), float32
- "stack"   — 3D numpy array (N, ny, nx), float32
- "volume" — 3D numpy array (nz, ny, nx), float32
- "fourier" — 2D complex, output of numpy.fft.rfft2
- "scalar" — int / float / str / bool
- "string" — text, wireable
- "doc"   — list[list[float]], one row per line

Ports mini-syntax

Both inputs and outputs use "name:kind[,name:kind]*". A trailing "?" on the kind marks that port as optional (no error if unwired).

```
# example: three inputs (one optional) and two outputs
inputs  = "a:image,b:image,mask:image?"
outputs = "cc:image,peak:scalar"
```

Param tuple shorthand

Each entry in the params list is either a Param object or a tuple. The tuple form is: (name, kind, default) or (name, kind, default, choices, min, max). Kind ∈ {'float', 'int', 'string', 'text', 'select', 'bool'}.

```
params = [
    ("sigma",   "float", 1.0, None, 0.0, 20.0),
    ("mode",    "select", "gauss", ["gauss", "box"]),
    ("verbose", "bool", False),
]
```

For richer forms — placeholders, wireability overrides, conditional visibility via depends_on — construct a Param directly:

```
from backend.plugin_api import Param

Param("batch", "bool", True),
Param("csv",   "string", "",
      wireable=True,
      depends_on=("batch", True),
      placeholder="phi,theta,psi per line"),
```

4. Adding a file format

The whole app reads files via read_array(path) and writes via write_array(path, arr). Both dispatch on the file extension. So a plugin that registers a new reader/writer immediately flows through io.load, io.save, io.load_stack, and the auto-save layer — no other changes needed.

Example: TIFF support (if you have tifffile installed)

```
"""tiff_support.py — drop in backend/plugins/ or ~/.pyspider/plugins/"""

import numpy as np
from pathlib import Path
import tifffile

from backend.plugin_api import register_reader, register_writer

def _read_tif(p: Path) -> np.ndarray:
    arr = tifffile.imread(str(p))
    return np.asarray(arr, dtype=np.float32)

def _write_tif(p: Path, data: np.ndarray) -> None:
    tifffile.imwrite(str(p), data.astype(np.float32))

for ext in (".tif", ".tiff"):
    register_reader(ext, _read_tif)
    register_writer(ext, _write_tif)
```

What you get for free

- io.load with a .tif path now returns the image.
- io.save with a .tif path writes TIFF.
- The auto-save layer honours whatever extension you set in the Inspector's save row.
- GET /api/plugins reports .tif in the readers/writers list.

5. Node with multiple outputs & wired params

This node computes both the mean and standard deviation of an image, exposing them as separate scalar outputs so downstream nodes can wire either one into their own parameters.

```
import numpy as np
from backend.plugin_api import register, N, h, d

def _fn_stats(image):
    a = h(image).astype(np.float32)
    return {
        "mean":  float(a.mean()),
        "sigma": float(a.std()),
        "mn":    float(a.min()),
        "mx":    float(a.max()),
    }

register(
    N("plugin.stats", "Simple stats (plugin)", "Custom",
      "image:image",
      "mean:scalar,sigma:scalar,mn:scalar,mx:scalar",
      [],  # no params
      "Emit mean, sigma, min and max of the input image.",
      _fn_stats),
)
```

Downstream usage

Wire "mean" into MA's "background" param, or "sigma" into TH M's threshold. Any scalar output can drive any wireable numeric param — this is the standard mechanism for data-dependent parameter tuning.

6. Working inside DO loops

Plugin nodes work inside the DO container without any special flags. If you want an accumulator-style node (like the built-in stack.collect that gathers each iteration's image into a final stack), set fn=None and handle the loop yourself — but that requires an engine hook. Recommended for most cases: keep your node stateless and let stack.collect do the accumulation.

Reading the DO iteration index

Inside a DO container, wire the DO node's `i` output into any int/float parameter of your plugin node. From your Python fn, the parameter simply arrives with the current iteration value.

```
def _fn_indexed(image, frame_id: int = 0):
    # frame_id becomes 0, 1, 2, ... on successive iterations
    # if DO.i is wired into it.
    ...
```

Streaming feedback

Long-running plugins can just take their time; the frontend shows them as 'running' (red, pulsing) until the fn returns. Per-frame progress within a single node is not (yet) surfaced in the UI; split into smaller iterations via DO if you want per-step feedback.

7. Testing & troubleshooting

Verify discovery

```
curl http://127.0.0.1:8000/api/plugins
# → {"plugins":[{"path":"…example_edge.py","ok":true,
#      "error":null,"nodes_added":1}], "readers":[...],
#      "writers":[...]}
```

Verify the node reaches the palette

```
curl http://127.0.0.1:8000/api/nodes | jq \
     '.nodes[] | select(.id | startswith("plugin."))'
```

Common issues

- ImportError at plugin load: check /api/plugins for the exception text. Fix your plugin's imports and restart uvicorn.
- Node id collides with a built-in: the built-in wins silently. Rename with a distinct prefix like plugin.mytool.
- "unknown node type" at execute time: the plugin loaded but the id in the frontend graph payload doesn't match. Reload the browser after adding new plugins.
- Auto-unwrap gotcha: 3D input to an image port is silently reduced to its first frame. If that surprises you, insert stack.index to be explicit.

Where to put your plugin

backend/plugins/	bundled with the distribution
$PYSPIDER_PLUGIN_DIR	colon-separated dirs; earliest wins
~/.pyspider/plugins/	user-scope, survives upgrades

8. Reference — plugin_api exports

Node building

`NodeSpec`	dataclass: id, label, category, inputs, outputs, params, doc, fn
`Port`	name + kind (+ optional=True for '?')
`Param`	name, kind, default, choices, min, max, wireable, depends_on, placeholder
`N(...)`	shorthand factory that parses port strings and param tuples into NodeSpec
`register(spec)`	add a NodeSpec to the plugin catalog

Format registry

`register_reader(ext, fn)`	fn takes Path, returns np.ndarray
`register_writer(ext, fn)`	fn takes (Path, np.ndarray), writes to disk
`read_array(path)`	dispatches on extension, returns numpy array
`write_array(p, a)`	dispatches on extension, writes to disk
`readers()`	list of registered read extensions
`writers()`	list of registered write extensions

Runtime helpers

`h(x)`	to-host: move x from cupy/numpy to numpy (CPU)
`d(x)`	to-device: move x to whatever backend is active (numpy or cupy)
`xp`	the current array namespace (backend.xp module)